

Embedded C-Programming: Pointer, Type Casting

Young-Keun Kim

Pointer

What are Pointers?

- A **pointer** is a variable whose value is the address of another variable
- i.e., direct address of the memory location.
- Pointers are the basis for data structures

(a) Define a pointer variable

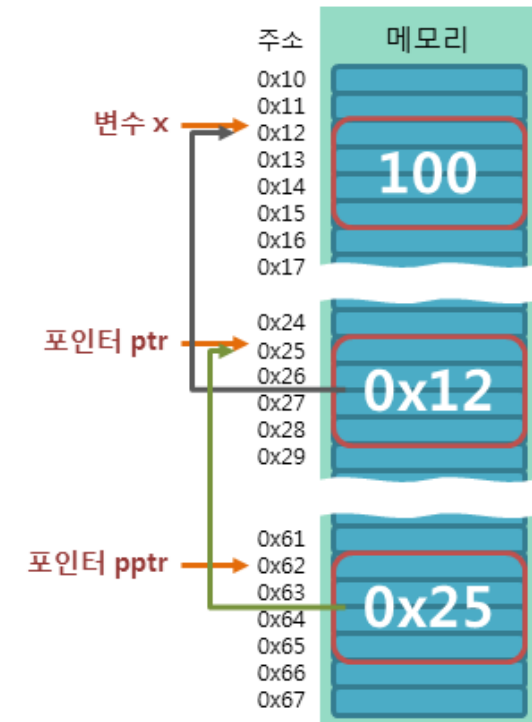
(b) Assign the address of a variable to a pointer and

(c) Access the value at the address available in the pointer variable

```
int *ptr;  
ptr = &x;  
int value = *ptr
```

```
int x = 100;    // 변수의 선언  
int *ptr = &x;  // 포인터의 선언  
int **pptr = &ptr; // 포인터의 참조
```

```
int main() {  
    int x = 100;  
  
    // Pointer Define  
    int *ptr;  
  
    // Pointer Assignment  
    ptr = &x; // store address of var in pointer variable  
  
    // Pointer Access  
    printf("Value of *ptr: %d", *ptr);  
  
    return 0;  
}
```



Pointer

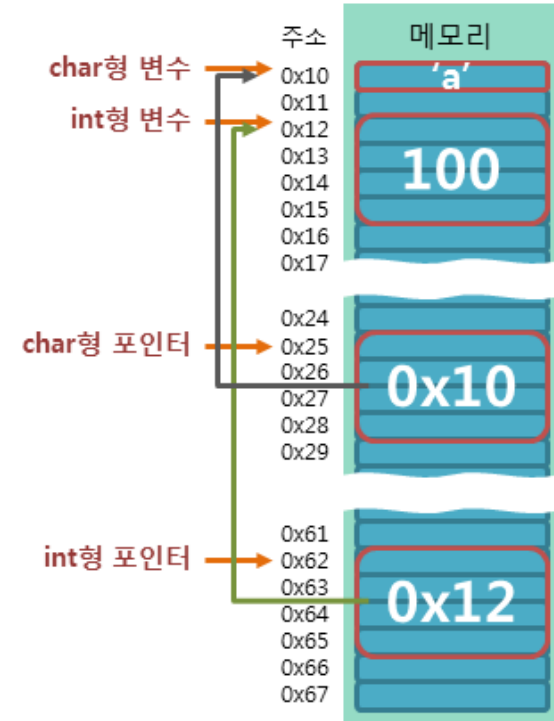
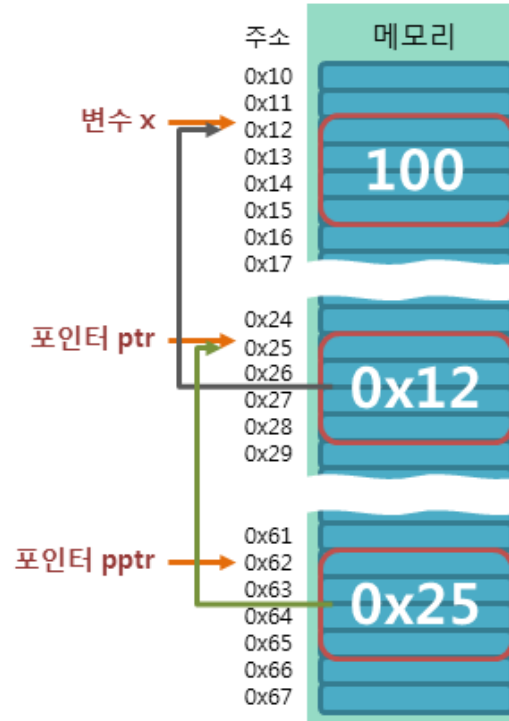
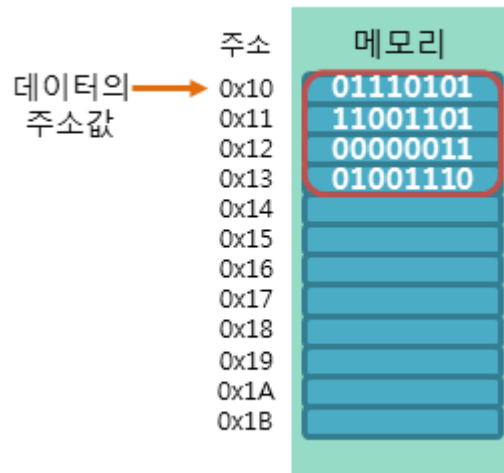
- 32-CPU: 1 word = 4 bytes (32bit)

- Memory: byte units
- Int : 4 byte
- Pointer variable: 4 byte

```
int n = 100; // 변수의 선언
int *ptr = &n; // 포인터의 선언
```

```
int x = 100; // 변수의 선언
int *ptr = &x; // 포인터의 선언
int **pptr = &ptr; // 포인터의 참조
```

```
int x = 100; // 변수의 선언
char y = 'a'; // 변수의 선언
int *ptr = &x; // 포인터의 선언
int *pptr = &y;
```



Pointer

● Examples:

Example 1

```
uint32_t a, b;
int main(void) {
    a = 3; b = 4;
    swap(&a, &b);
}

void swap(uint32_t* a, uint32_t* b) {
    uint32_t t;
    t = *a;
    *a = *b;
    *b = t;
}
```

Example 2

```
char str1[13] = "ARM Assembly";
char str2[13];
strcpy(str2, str1);

void strcpy(char* dest, char* source) {
    while (*source) {
        *dest = *source;
        dest++;
        source++;
    }
    *dest = *source; // termination 0
}
```

Example 3

```
uint16_t const Prime[3] = { 1,2,3 };
uint16_t const* Pt;      // const does NOT indicate pointer is fixed. Refers to const data
Pt = Prime;              // &Pt=Prime[0]
```

Type Casting of Pointer Variable

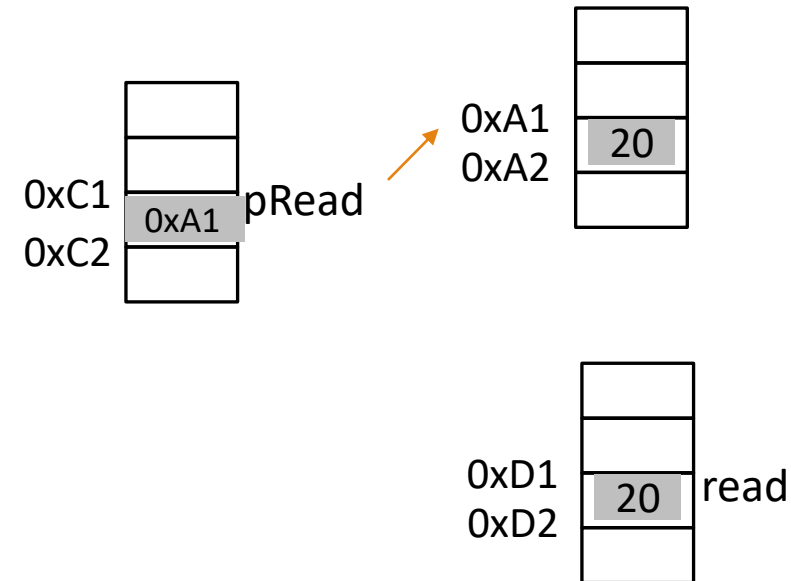
- Type Casting: To tell the size of variables which it is pointing

```
uint8_t *pRead= (uint8_t*) 0xA1;
```

```
uint8_t* pRead;      // declare as a pointer
pRead = 0xA1         // points to address @0xA1

// However, @0xA, is the value 8bit? 32bit?
// Use typecast (uint8_t * ) force conversion to 8-bit type pointer

pRead = (uint8_t*)0xA1;
uint8_t read = *pRead; // read=20
```



Revisit to Structure

● Examples Structure in STM32F

```
typedef struct {  
    uint32_t MODER;    // Mode register  
    ...  
    uint32_t ODR;      // Output data register  
    ...  
} GPIO_TypeDef;
```

// Option 1: Variable of GPIO_TypeDef type.

GPIO_TypeDef PortA;

PortA.ODR= 15;

// Option 2: Pointer of GPIO_TypeDef type.

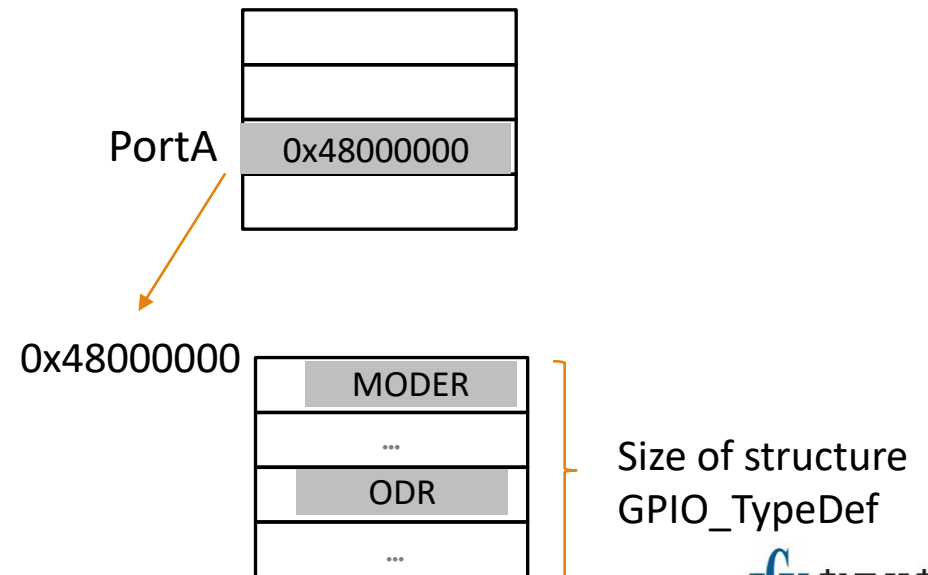
GPIO_TypeDef *PortA;

PortA = (GPIO_TypeDef*) 0x48000000;

PortA->ODR= 15;

#define PortA ((GPIO_TypeDef *) 0x48000000)

PortA->ODR= 15;



Revisit to Structure

● Examples Structure in STM32F

```
typedef struct {  
    uint32_t MODER;    // Mode register  
    ...  
    uint32_t ODR;      // Output data register  
    ...  
} GPIO_TypeDef;  
  
void GPIO_init(GPIO_TypeDef *Port) {...};
```

// What are the differences?

(1) #define GPIOA (0x48000000)

GPIOA->ODR= 15; // error

(2) #define GPIOA ((GPIO_TypeDef *) 0x48000000)

GPIOA->ODR= 15;
GPIO_init(GPIOA);

(3) #define GPIOA (*(GPIO_TypeDef *) 0x48000000)

GPIOA.ODR= 15;
GPIO_init(&GPIOA);